

Входная диагностика

Сначала пара вопросов о вас,
потом восемь задач

На оценку не влияет

Не знаете — так и отмечайте.



forms.gle/YCX1xesDKCeWfz9W9

Фамилию можно не указывать

Программирование на Python для бизнес-аналитики

Магистратура «Бизнес-аналитика и цифровые решения в финансах» · ВШБ НИУ ВШЭ

Рабочая среда, Git и идиоматика для аналитика

Дмитрий Тихомиров · 5 сентября 2026

«У меня работает» — это не результат.

Результат — это когда работает
у того, кто **откроет ваш проект**
через полгода.

Четыре способа потерять готовый расчёт

- **У коллеги другая версия библиотеки.**
Ваш код у него падает. Или, хуже, не падает и считает иначе.
- **Нужный файл лежал только у вас на диске.**
В репозитории его нет, потому что «он же большой».
- **Ноутбук выполнялся не сверху вниз.**
У вас цифры сошлись. С чистого запуска — не сойдутся.
- **Пароль был вписан в код, и его сменили.**
А заодно он уехал в историю репозитория и остался там навсегда.

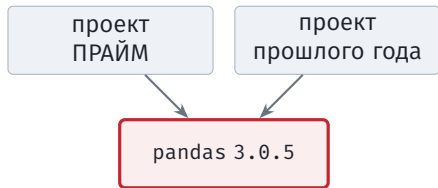
Ни одна из четырёх — не про знание Python.
Все четыре — про то, о чём эта лекция.

Рабочая среда

чтобы расчёт пережил ваш ноутбук

Виртуальное окружение: зачем оно вообще

Без окружений



один набор пакетов на всё:
обновили для одного — сломали другой

С окружениями



у каждого проекта своя папка .venv:
соседи друг друга не видят

Окружение — это *папка внутри проекта*, а не настройка компьютера.
Удалили `.venv` — ничего не потеряли: она собирается заново одной командой.

Чем управлять зависимостями

	Что даёт	Чего не хватает
<code>pip + venv</code>	Есть везде, ничего ставить не надо	Нет lock-файла: <code>requirements.txt</code> фиксирует не всё и не у всех одинаково
<code>poetry</code>	Настоящий lock-файл, зрелый инструмент	Медленный; Python себе не ставит
<code>conda</code>	Тянет и не-Python зависимости	Тяжёлый, свои каналы и свои конфликты
<code>uv</code>	Lock, окружение и сам Python — одной командой. На порядок быстрее остальных	Молодой: на работе встретится не везде

Курс идёт на `uv`. Принцип общий для всех четырёх: **окружение отдельно, версии зафиксированы**. Встретите на работе `poetry` — переучиваться не придётся.

Проект с нуля — четыре команды

```
$ pip install uv                                # один раз на компьютер

$ uv init --python 3.12 prime-monitor          # создать проект
$ cd prime-monitor                             # перейти в папку проекта
$ uv add "pandas>=3.0"                         # добавить зависимость
Creating virtual environment at: .venv
Installed 4 packages in 58ms
+ numpy==2.5.2
+ pandas==3.0.5
+ python-dateutil==2.9.0.post0
+ six==1.17.0
$ uv run main.py                               # запустить в этом окружении
3.0.5
```

Просили один пакет — встало четыре: у pandas свои зависимости.

Сам uv ставится один раз и не в проект, а в систему. Через pip — способ, одинаковый везде; на macOS удобнее `brew install uv`, есть и официальный установщик — см. docs.astral.sh/uv.

Пятая команда — для того, кто *клонировал* ваш проект: `uv sync`.

Два файла, которые всё решают

pyproject.toml — что я попросил

```
[project]
requires-python = ">=3.12"
dependencies = [
    "pandas>=3.0",
]
```

Пишется руками или командой `uv add`.
Диапазоны. Читается человеком. **9 строк**.

uv.lock — что реально встало

```
name = "pandas"
version = "3.0.5"
sdist = { url = "...",
    hash = "sha256:dca3734d6ab7c9..." }
```

Генерируется. Точные версии и хеши.
Руками не правят. **171 строка на 4 пакета**.

В git идут оба. Коллега запускает **uv sync**, а не `uv add` заново, — и получает *ваши* версии, а не сегодняшние. Это и есть воспроизводимость, и её проверяют на ДЗ-1.

Как выглядит проект аналитика

Что лежит	Зачем	В git?
pyproject.toml	Что за проект и что ему нужно	да
uv.lock	Точные версии всего установленного	да
.python-version	Версия интерпретатора	да
.gitignore	Что в репозиторий не пускать	да
README.md	Как это запустить у себя	да
.env.example	Имена переменных, без значений	да
src/prime/	Код: загрузка, метрики, запуск	да
notebooks/	Черновики и разведка	да
.env	Пароли и токены	нет
data/	Выгрузки из учебной базы	нет
.venv/	Само окружение — собирается заново	нет

Ровно эта структура — в шаблоне репозитория, который вы получите на семинаре.

Ноутбук и модуль — разные жанры

	Ноутбук	Модуль .py
Для чего	Думать: посмотреть, попробовать, отбросить	Поставить результат: то, что будет работать без вас
Сильная сторона	Видно данные и график рядом с кодом	Импортируется, запускается по расписанию, построчно видно, что изменилось
Слабая сторона	Скрытое состояние и нечитаемая история изменений	Чтобы что-то увидеть, надо это специально напечатать

Это не «ноутбук для новичков, модуль для взрослых».

Разведка живёт в ноутбуке, поставка — в модуле. Переезд из первого во второе —
тема 09.

Ноутбук помнит то, чего вы не видите

In [1] `payments = pd.read_csv(...)`

In [7] `payments = payments[payments.amount > 0]`

In [3] `revenue = payments.amount.sum()`

In [5] `ячейка удалена: rate = 0.87`

In [12] `print(revenue * rate)`

Порядок на экране — **сверху вниз**.
Порядок исполнения — **1, 7, 3, 12**.
Выручка посчитана до фильтра.

Ячейки нет, а переменная `rate` жива в памяти. У вас работает, у проверяющего — `NameError`.

Оба случая закрывает одно правило: перед тем как показать ноутбук кому-либо — **Restart Kernel and Run All Cells**. Не проходит целиком — показывать нечего.

Где всё это писать

	Хорош для	Плох для
JupyterLab	Разведки: данные, график и код на одном экране	Проекта из многих файлов, поиска по коду, работы с git
VS Code	Всего сразу: и ноутбуки, и модули, и git, и терминал	Ничего критичного. Стартовая рекомендация курса
PyCharm	Крупного кода: рефакторинг и отладчик сильнее	Тяжелее; часть возможностей платная

Выбор редактора — вопрос привычки, и спорить о нём мы не будем.
Требование курса ровно одно: проект запускается из терминала командой `uv run`, а не «кнопкой в моей среде».

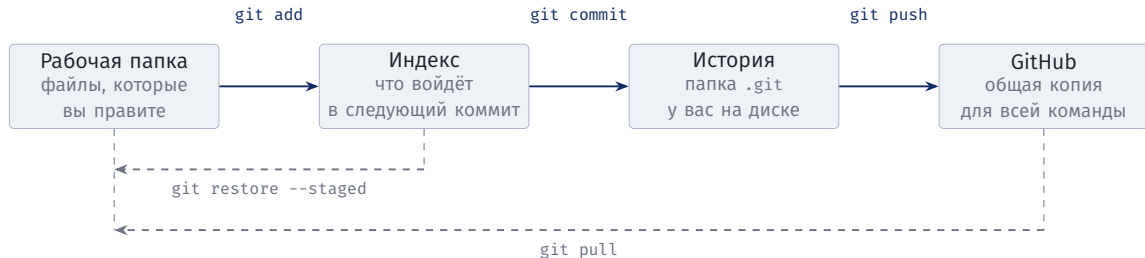
Git

история решений, а не папка «финал_v3_финал»

Git — это не бэкап.

Это ответ на вопрос
«кто, когда и зачем»
поменял вот эту строку.

Четыре места, где живёт ваш файл



Почти все «я всё сломал в git» — это непонимание, в каком из четырёх мест лежит файл.

Ровно на этот вопрос и отвечает `git status` — им сейчас и займёмся.

git status показывает, где сейчас файл

```
$ git status
On branch feature/weekly-metrics
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   src/prime/metrics.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    data/
```

Changes to be committed — файл в индексе, войдёт в следующий коммит

Changes not staged — правки есть, но живут только в рабочей папке

Untracked — git про этот файл ещё вообще не знает

Запускайте `git status` чаще, чем кажется нужным:

он не только отвечает, где файл, но и в скобках называет команду, которая его двигает.

Ежедневный цикл: шесть команд

```
$ git pull                                # забрать чужие изменения
$ git switch -c feature/weekly-metrics   # своя ветка под задачу
$ git status                             # что изменилось
$ git add src/prime/metrics.py           # выбрать, что войдёт в коммит
$ git commit -m "считаем выручку без учёта возвратов"
$ git push -u origin feature/weekly-metrics
                                           # дальше — pull request на GitHub
```

- `git add .` кладёт в коммит *всё изменённое разом* — включая правку в соседнем файле, которую вы делали «на посмотреть» и сохранять не собирались. **Называйте файлы явно**
- Имена веток и файлов — **латиницей**. Кириллица в ветках ломается в половине инструментов
- Застрали — `git status`. Он почти всегда подсказывает следующую команду

diff: что именно изменилось

```
$ git diff
diff --git a/src/prime/metrics.py b/src/prime/metrics.py
index 34a798a..b170220 100644
--- a/src/prime/metrics.py
+++ b/src/prime/metrics.py
@@ -1,2 +1,3 @@
 def revenue(payments):
-     return payments["amount"].sum()
+     paid = payments[payments["status"] == "ok"]
+     return paid["amount"].sum()
```

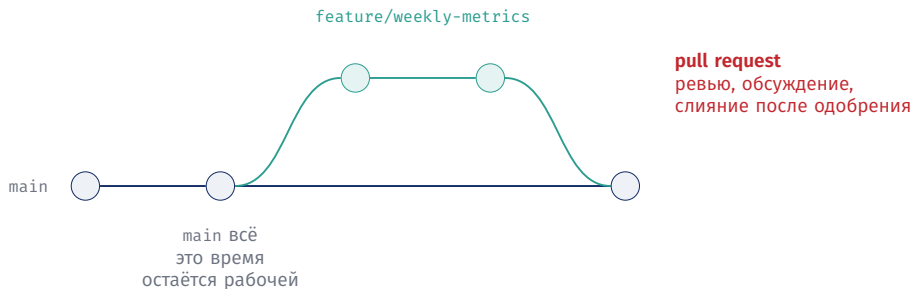
- - — строка, которая *была*. + — строка, которая *стала*. Всё остальное git не печатает: в файле может быть тысяча строк
- @@ -1,2 +1,3 @@ — с какой строки файла посмотреть и сколько их там теперь
- Ровно это видно в pull request: **ревью идёт по diff**, а не по файлу целиком

Что такое хороший коммит

Так не надо	Так надо
40 файлов одним коммитом «работа за вторник»	Один коммит — одно законченное изменение
"fix", "update", "."	"убрал двойной счёт в витрине клиента"
"поменял округление в revenue" — пересказ того, что и так видно в diff	"округляем выручку до копеек: расхотились с бухгалтерией" — зачем
Коммит, после которого ничего не запускается	Каждый коммит оставляет проект в рабочем состоянии

Проверка на вменяемость: сообщение должно быть полезно вам же через полгода, когда вы будете искать, где сломалась цифра.

Ветка и pull request



Pull request нужен не для церемонии. `main` никогда не сломан, обсуждение привязано к строкам, и работу **кто-то посмотрел** до того, как ею начали пользоваться.

Конфликт — это не авария

```
$ git merge feature/paid-only
Auto-merging metrics.py
CONFLICT (content): Merge conflict in metrics.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
def revenue(payments):
<<<<<<< HEAD
    return payments["amount"].sum().round(2)      # что было у вас
=====
    paid = payments[payments["status"] == "ok"]   # что пришло из ветки
    return paid["amount"].sum()
>>>>>> feature/paid-only
```

Конфликт возникает, когда двое поменяли *одни и те же строки*: git отказывается решать за вас. Убрать три строки-маркера, оставить правильный код (часто — обе половины), git add, git commit.

Что не кладут в репозиторий

Никогда

- `.env`, пароли, токены, ключи
- Выгрузки данных: `*.csv`, `*.parquet`, дампы базы
- `.venv/` — она собирается из `lock`-файла
- `.ipynb_checkpoints/`, `__pycache__/`

Вместо этого

- `.env.example` с именами переменных и пустыми значениями
- Код, который данные *достаёт*, и описание, откуда
- `uv.lock`
- `.gitignore`, написанный до первого коммита

Данные учебной базы — не ваши, чтобы их публиковать. На работе выгрузка клиентских данных в публичный репозиторий — **не «ошибка новичка»**, а инцидент информационной безопасности.

.gitignore пишется в первый день

```
# Секреты
.env
*.key

# Данные: учебная база живёт на сервере,
# локальные выгрузки в репозиторий не попадают
data/
*.csv
*.parquet

# Служебное
.venv/
__pycache__/
.ipynb_checkpoints/
```

Это фрагмент .gitignore *этого* репозитория — можно использовать.

Написанный заранее, он стоит две минуты. Написанный после аварии — не помогает.

Ноутбуки в git: отдельная боль

- `.ipynb` — это **JSON**, а не текст. В diff вы увидите не «поменялась формула», а перемешанные служебные поля
- Внутри лежат **результаты выполнения**: таблицы с данными и картинки в base64. Ноутбук на 30 мегабайт — обычное дело

Правило курса. Ноутбук сдаётся **исполненным сверху вниз** — проверяющий должен видеть ваши цифры. Но перед коммитом уберите тяжёлые выводы: гигантские таблицы и картинки на мегабайты. Код должен читаться в diff.

На семинаре настроим автоматическую очистку, чтобы не помнить об этом руками.

Git ставится руками, а не слушанием

Тренажёр **Git Exercises**

- 23 задачи на настоящем репозитории. Решение проверяется автоматически, командой `git verify`
- **Первые одиннадцать — до `fix-tyro` включительно — советую пройти до семинара**
- Дальше — по желанию. Там начинается переписывание истории, оно вам понадобится не завтра

Часть задач — про то, чего на лекции не было. Так и задумано: к каждой есть подсказка.



gitexercises.fracz.com

Когда всё сломалось

и это нормальная часть работы

Трейсбек читают не сверху и не снизу

```
Traceback (most recent call last):
  File "~/prime-monitor/run.py", line 12, in <module>
    print(revenue_by_channel(payments))
  File "~/prime-monitor/src/prime/metrics.py", line 5, in revenue_by_channel
    paid = payments[payments["status"] == "ok"]
           ~~~~~^~~~~~
  File "~/venv/.../pandas/core/frame.py", line 4378, in __getitem__
    indexer = self.columns.get_loc(key)
  File "~/venv/.../pandas/core/indexes/base.py", line 3648, in get_loc
    raise KeyError(key) from err
KeyError: 'status'
```

- **Последняя строка** — что случилось: `KeyError: 'status'`
- **Последняя строка с вашим файлом** — где чинить. Ниже неё почти всегда чужой код
- `~~^^` под выражением — Python показывает, какая именно часть строки упала

Если написано «The above exception was the direct cause of the following exception» — читайте *нижний* блок: верхний обычно внутренности библиотеки.

Пять ошибок вашей первой недели

Что вы увидите	Что это на самом деле
<code>ModuleNotFoundError:</code> <code>No module named 'pandas'</code>	Пакета нет в этом окружении. <code>uv add pandas</code> , запуск через <code>uv run</code>
<code>FileNotFoundError:</code> <code>'data/payments.csv'</code>	Путь относительный, а рабочая директория другая. Проверьте, откуда запускаете
<code>KeyError: 'status'</code>	Нет такой колонки: опечатка, регистр, пробел в названии — или её переименовали выше по коду
<code>ValueError: could not convert string to float: '1 200'</code>	Тип не тот, что вы думали. В данных разделитель разрядов тема 03
<code>AttributeError: 'NoneType'</code> <code>object has no attribute 'sum'</code>	Функция вернула <code>None</code> . Классика: метод отработал «на месте» и ничего не вернул

Все пять — не про Python. Про окружение, пути, данные и типы.

Как спросить, чтобы вам ответили

Не работает

- «У меня ошибка, помогите»
- Фотография экрана с телефона
- Кусок трейсбека — «остальное там неважное»
- «Вчера работало»

Работает

- Что делал, что ожидал, что получил
- **Полный трейсбек текстом**, а не картинкой
- Минимальный кусок кода, который воспроизводит
- Версии: `uv run python -V`, `uv.lock`

Половина вопросов, сформулированных по правому столбцу, отвечает себе сама по дороге. Вторая половина получает ответ за минуты вместо часов.

Тот же список — про запрос к ИИ-ассистенту. Подробно об этом тема 12.

К семинару 9 или 12 сентября

Поставить заранее

- Python **3.12+**
- uv — по инструкции с сайта
`docs.astral.sh/uv` — это адрес документации, а не команда
- Git
- Аккаунт на GitHub

Проверка, что всё встало:

```
uv --version  
git --version
```

Что будет на семинаре

- Свой репозиторий из шаблона, ветка, pull request
- Конфликт — устроим намеренно и разберём
- Систематизация Python: коллекции, распаковка, исключения, аннотации
- **Выдача ДЗ-1**

По желанию, но очень советую: тренажёр `gitexercises.fracz.com` до задачи `fix-typo`.

ДЗ-1 — окружение, репозиторий и воспроизводимый запуск расчёта. Выдача **12.09**, дедлайн **19.09, 23:59** — общий для обеих групп. Приём со снижением балла до 26.09.

Что унести с этой пары

- В `git` идут и `pyproject.toml`, и `uv.lock`
Иначе у того, кто клонирует ваш проект, встанут не ваши версии, а сегодняшние
- Ноутбук показывают только после **Restart and Run All**
- Застряли в `git` — `git status`
- В трейсбеке ищут **последнюю строку со своим файлом**

Это была самая инструментальная пара курса — дальше начинаются данные. Со следующей темы работаем с настоящей базой ПРАЙМ, и инструменты **уходят из центра внимания**.



Вопросы — в чат курса или в личку